

Quiz #3 报告：Quad SSA 过程信息收集及信息解释

一、核心代码思路

本次在 HW7 的 `quadssa.cc` 中，于 SSA 转换的四个阶段分别收集诊断信息，并在每个函数转换完成后调用 `printSsaDiagSummary(funcdecl, g_diag)` 打印结果，具体思路如下所示：

(1) 先对候选的 ϕ 块进行分析，对每个变量运行迭代支配边界闭包，记录所有可能需要插入 ϕ 的基本块，对应 `candidatePhiBlocksByVar` (2) 然后对 ϕ 过滤与放置，仅当变量在 **2 个及以上** 基本块中有定义时，才真正插入 ϕ 函数，最终 ϕ 由 `printSsaDiagSummary` 扫描得到 `actual_phi` (3) 变量改名，每次产生新版本，但是注意不含首次赋值的 `v0`，时，记录版本号及其所在基本块，对应 `createdVersionBlocksByVar` (4) 删除无用 ϕ ，在 `cleanupUnusedPhi` 中删除未被使用的 ϕ 时，记录被消除的版本，对应 `eliminatedVersionBlocksByVar`

二、test示例

所选test与变量

- 测试用例：`quadtest6`
- 分析变量：`t103` 具体解释如下：

```
SSA_TEMP_DIAG temp=t103 candidate_phi=[L102]
```

容易发现这个变量有 `candidate_phi=[L102]`，但**没有** `actual_phi` 字段，说明最终 SSA 程序中未放置 ϕ 函数。但是同函数中的循环变量 `t101`，它其实对应源码中的 `i`。它的输出是：

```
SSA_TEMP_DIAG temp=t101 candidate_phi=[L102] actual_phi=[L102] created=[v1@{L102},v2@{L107}]
```

既有候选块，也在 L102 实际放置了 ϕ 。

`quadtest6.fmj` 的 `main` 函数在编译到 Quad 后，上面提到的相关变量对应关系应该是这样的：

- `t101`：循环变量 `i`
- `t103`：`length(a)` 的结果，在循环条件处使用

画一个简单的控制流结构就是：

```

L108 ( 初始化数组、i=0 )
    ↓
L102 ( 循环头：判断 i < length(a) ) ←———
    ↓ true                ↓ false
L103 ( 循环体入口 )        L104 ( return )
    ...
L107 ( putchar、i=i+1 ) —————

```

所以L102 是循环头，具有来自 L108（首次进入）和 L107（回边）两条前驱路径，是一个**汇合点**。

回答问题

为什么**t103**有**candidate phi (L102)**？在 SSA 转换的**第一步**，对每个变量运行迭代支配边界算法，具体的步骤应该是：

1. 收集**t103**的定义块：**t103** 由 `LOAD t103 <- Mem(t100:ptr)` 定义，该语句位于 L102，因此 `def blocks = {L102}`。
2. 以 L102 为起点，查支配边界：`DF(L102) = {L102}`，发现L102是循环头，L107的回边使L102落在自身的支配边界上。
3. 将L102加入**t103**的候选 ϕ 块集合。

所以**candidate_phi=[L102]** 表示，按照支配边界分析，**t103** 可能需要在 L102 处插入 ϕ 以合并来自不同路径的值。同一循环头 L102 也是其他多个变量的候选块，因为它们各自的 def 块通过 IDF 闭包同样可达 L102。

为什么 **t103** 最终没有放置 phi？

在 SSA 转换的第二步中，需要进一步去判断候选块上的 ϕ 是否真正必要。应该使用的过滤条件是：**当变量在 2 个及以上不同基本块中有定义时，才插入 ϕ 函数**。

所以对 **t103** 而言：

- 在整个函数的静态控制流图中，**t103**只在 L102 一个基本块中被定义，因为每次循环迭代重新执行同一条 `LOAD` 语句。
- 虽然 L102 是循环汇合点，但 **t103** 并不存在"来自 L108 和 L107 两条不同路径、各自产生不同静态定义"的情况。
- 每次进入 L102 时，`LOAD t103 <- Mem(t100:ptr)` 都会**重新计算**数组长度；该值不依赖回边携带的旧版本，无需通过 ϕ 合并。

因此 `defBlocks[t103].size() == 1`，过滤后不插入 ϕ 。

最终quadtest6的SSA程序中，L102 处 **t103** 仍然应该是普通的LOAD，对应语句：

```
LOAD t10300:int <- Mem(t10000:ptr); def: t10300 use: t10000
```

但是**t101** 因 def 块为 `{L108, L107}`，有两处赋值，符合上述的过滤条件，就应该对应：

```
PHI t10101:int <- Phi([t10102, L107], [t10100, L108]); ...
```

三、实验结果

对老师给出的9个test进行测试，最终提取出的9个diag在终端的输出与老师所给的参考输出完全一致，并且保证-ssa.quad仍然正确。